

Lecture 6: Advanced Procedures

Table of contents

1	Objectives	2
2	Advanced Procedures	2
2.1	Introduction	2
2.2	Stack Frame	4
2.2.1	Accessing Arguments and Local Variables	6
2.2.2	Creating and Destroying a Stack Frame	6
2.3	32-bit Calling Conventions	11
2.3.1	C Calling Conventions (cdecl)	11
2.3.2	STDCALL Calling Convention	14
2.4	LEA Instruction	15
2.4.1	Flags Affected	17
2.5	ENTER and LEAVE Instructions	17
3	Recursion	19
3.1	Introduction	19
3.2	Key Concepts	19
3.3	Recursive Factorial Example	19
3.3.1	NASM Implementation	20
3.4	Visualizing Recursion: The Stack for 3!	21
3.5	Recursive Sum Example	24
3.6	Integer to Binary String Example	25
3.6.1	The Recursive Strategy	26

3.6.2	Tracing <code>print_binary(26)</code>	26
3.6.3	The Assembly Implementation	27

1 Objectives

In this lecture, you will learn:

1. How to pass arguments to procedures using the run-time stack.
2. How to define, create, and release a stack frame.
3. The rules and purpose of 32-bit calling conventions (`cdecl` and `stdcall`).
4. How to implement recursive procedures in assembly language.

2 Advanced Procedures

2.1 Introduction

In previous lecture, we passed arguments to procedures using registers. While fast, this method is limited by the small number of available registers. A more robust and scalable method is to use the run-time stack.

Consider this simple C program:

```
#include <stdio.h>

int getMin (int, int);

int main()
{
    int num1 = 10;
    int num2 = 8;
    int min;
```

```

min = getMin(num1, num2);

printf("The minimum of %d and %d is %d", num1, num2, min);

return 0;
}

int getMin (int a, int b)
{
    if (a<b)
        return a;
    else
        return b;
}

```

- The program consists of two functions `main()` and `getMin()`.
- The `main()` function invokes `getMin()` and **passes two arguments**: `num1` and `num2`. Precisely, the main function passes the **values** of `num1` and `num2`.
- How does this happen at the assembly level? This lecture will explain the mechanics of passing arguments using the run-time stack.

! Terminology

- The procedure that initiates the call is the **caller** (`main()` in this case).
- The procedure that is being called is the **callee** (`getMin()`).
- The values passed by the caller are **arguments** (`num1, num2`).
- The variables in the callee that receive these values are **formal parameters** or simply **parameters** (`a, b`).

2.2 Stack Frame

Most modern languages *pushes arguments on the stack* before calling a procedure. The callee then uses the stack to store its own local variables.

Definition

A **stack frame**, also known as an **activation record**, is a designated block of memory on the stack segment created for a single procedure call. It holds the passed arguments, the return address, and any local variables for that procedure.

In 32-bit mode, C/C++ functions and the Windows API rely heavily on the stack frame. A typical *stack frame* is built in the following order:

1. **Argument(s)**: The caller pushes arguments onto the stack.
2. **Return Address**: The CALL instruction automatically pushes the return address.
3. **The old EBP register**: The callee pushes the caller's **stack frame pointer** (EBP) so it can be restored later.
4. **Local Variables**: The callee allocates space on the stack for its local variables by subtracting the required number of bytes from the stack pointer (ESP). For example, `sub esp, 16` reserves 16 bytes of stack space for local variables.
5. **Registers**: The callee pushes any other registers it will modify.

Figure 1 shows the main components of a typical stack frame that should be established when calling a procedure.

Figure 2 shows a stack frame for a procedure that receives three arguments and has two local variables. For simplicity, all variables are doubleword (4 bytes).

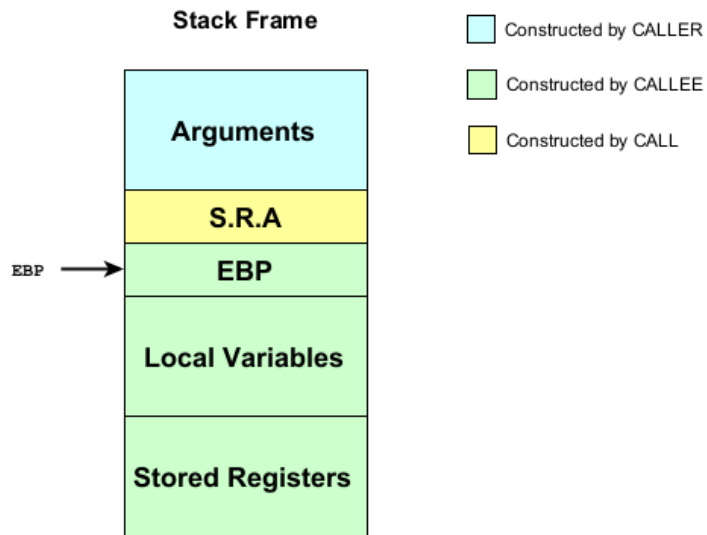


Figure 1

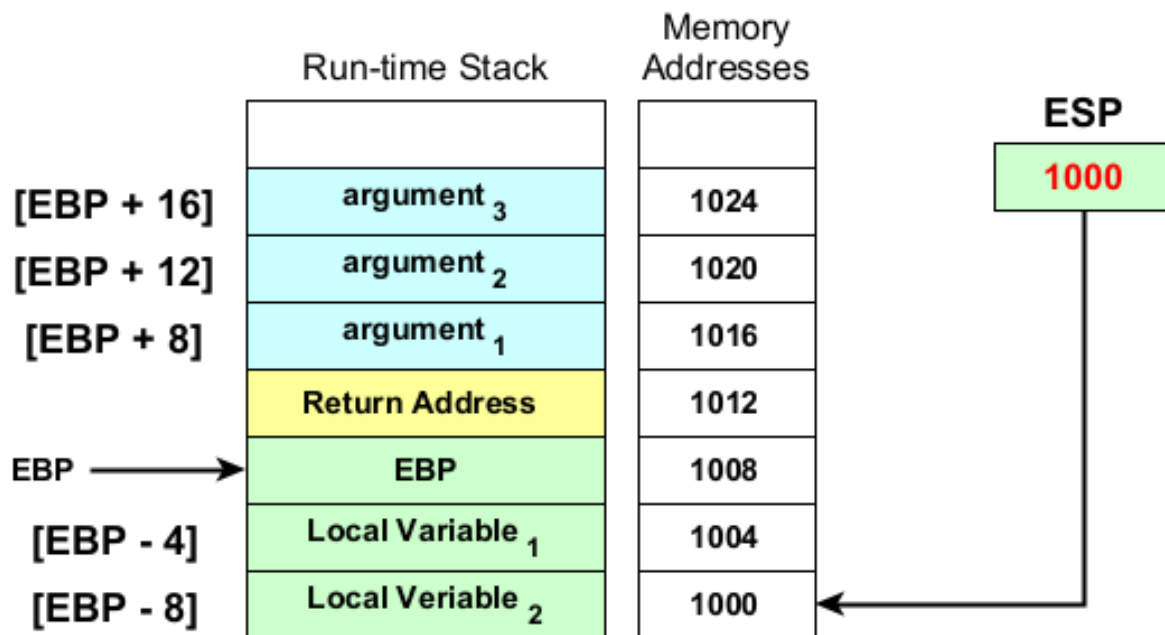


Figure 2

2.2.1 Accessing Arguments and Local Variables

- The standard way to access passed arguments and local variables inside assembly program is by using **base+displacement addressing mode**.
- The base address is EBP register and the displacement is the offset from the base register.
- Arguments are located at *positive offsets* from EBP. For example, the address of the first argument is EBP+8. The second argument is located at address EBP+12.
- Local Variables are located at *negative offsets* from EBP. For example, the first local variable is located at address EBP-4.
- Please refer to Figure 2 for the details of memory addresses of the other variables.

2.2.2 Creating and Destroying a Stack Frame

- Creating a complete stack frame is a collaborative process between the caller and the callee.
- The caller begins the process by pushing the procedure's arguments onto the stack.
- The CALL instruction continues the process by pushing the return address
- The callee then finalizes the frame creation by saving the old EBP, setting the new EBP as the frame pointer, and allocate space for local variables. These steps are known as **Prologue**.
- The callee, at the end, destroys its stack frame by deleting the local variables and restoring the old value of EBP. These steps are known as **Epi-logue**.
- Here is a typical procedure skeleton that uses a stack frame:

```

; --- Procedure skeleton ---
procedure_name:
    ; --- Prologue ---
    push    ebp        ; save EBP in stack
    mov     ebp, esp    ; set EBP points to its old value
    sub     esp, <n>    ; allocate n bytes for local
                        ; variables

    .
    .
    .    <procedure body>
    .
    .
    ; --- Epilogue ----
    mov     esp, ebp    ; delete local variables
    pop     ebp        ; restore old EBP value
    ret

```

Example 2.1 (AddTwo Procedure). Here is a C function and its assembly equivalent which uses a stack frame. The procedure has two parameters but no local variables.

```

int AddTwo (int x, int y)
{
    return (x+y);
}

```

```

AddTwo:
    ; Prologue
    push    ebp
    mov     ebp, esp

    ; Body
    mov     eax, [ebp + 12] ; second parameter
    add     eax, [ebp + 8]  ; first parameter

    ; Epilogue

```

```
    pop    ebp
    ret
```

To make our code more readable, we can define symbolic constants to represent the formal parameters.

```
AddTwo:

#define x_param DWORD [ebp + 8]
#define y_param DWORD [ebp + 12]
    ; Prologue
    push    ebp
    mov     ebp, esp
    ; Body
    mov     eax, y_param
    add     eax, x_param
    ; Epilogue
    pop     ebp
    ret
```

The main procedure would call AddTwo procedure like this:

```
    push    6        ; push second argument
    push    5        ; push first argument
    call    AddTwo
    add     esp, 8    ; clean up 2 arguments
```

Example 2.2 (MinThree Procedure). Here is another example where the callee function has three parameters and one local variable.

```
int MinThree (int a, int b, int c)
{
    int min = a;

    if (b < min) min = b;
```



```

    if (c < min) min = c;

    return min;
}

int main()
{
    printf("Minimum number is %d\n", MinThree(15, 10, 13));

    return 0;
}

```

The equivalent program in assembly language is

```

        global _main
        extern _printf

        section .data
prnt_fmt    db      "Minimum number is %d.", 13, 10, 0

        section .code

;-----
; MINTHREE Procedure
;-----
MinThree:
%define     a_param DWORD [ebp + 8]
%define     b_param DWORD [ebp + 12]
%define     c_param DWORD [ebp + 16]
%define     min_loc  DWORD [ebp - 4]

        ; prologue
        push    ebp
        mov     ebp, esp
        sub     esp, 4                ; allocate memory for local variable

        ; set a as the min value

```

```

        mov     eax, a_param
        mov     min_loc, eax
        ; if b < min => min = b
        mov     eax, b_param
        cmp     eax, min_loc
        jnl     next1
        mov     min_loc, eax
next1:   mov     eax, c_param
        cmp     eax, min_loc
        jnl     next2
        mov     min_loc, eax
next2:   mov     eax, min_loc

        ; epilogue
        mov     esp, ebp           ; destroy local variable
        pop     ebp
        ret

;-----
; MAIN Procedure
;-----
_main:
        push    15
        push    10
        push    13
        call    MinThree
        add     esp, 12

        push    eax
        push    prnt_fmt
        call    _printf
        add     esp, 8

        xor     eax, eax
        ret

```

2.3 32-bit Calling Conventions

- Calling conventions can be described as a **standardized set of rules that act as an interface** between a caller and a callee. It defines:
 - The order in which arguments are passed.
 - How parameters are passed (pushed on the stack, placed in registers, or a mix of both)
 - Which registers the callee function must preserve for the caller.
 - Who is responsible for cleaning up the stack (the caller or the callee).



Application Binary Interface (ABI)

An application binary interface (ABI) is an interface between two binary program modules. Basically, the interface consists of calling conventions, type representations and name mangling.

- In this course, we present the two most commonly used calling conventions for 32-bit programming in a Windows environment: **cdecl** and **stdcall**.

2.3.1 C Calling Conventions (cdecl)

The **cdecl** (C declaration) was established by the C programming language, and used by C and C++ programming languages.

- **Argument Order:** The caller pushes arguments onto the stack in reverse order (i.e., from right to left).
- **Return Value:** If the return values are integer values or memory addresses, they are put into the EAX register by the callee.
- **Stack Cleanup:** The caller cleans the stack after the function call returns.

The procedures in Example 2.1 and Example 2.2 are following cdecl convention.

Example 2.3 (cdecl convention). The following program consists of two files: `main.c` and `minthree.asm` files. The `main.c` file has the `main()` function written in C language. The `minthree.asm` file has the `MinThree` function written in assembly language. Since the later function uses cdecl convention, the `main()` function can invoke `MinThree` function without any problem.

FILE: `main.c`

```
#include <stdio.h>

int MinThree (int, int, int);

int main()
{
    int m = MinThree(3, 2, 1);
    printf("Minimum number is %d\n", m);
}
```

FILE: minthree.asm

```
global _MinThree

section .code

_MinThree:
%define a_param DWORD [ebp + 8]
%define b_param DWORD [ebp + 12]
%define c_param DWORD [ebp + 16]
%define min_loc DWORD [ebp - 4]

; prologue
push    ebp
mov     ebp, esp
sub     esp, 4           ; allocate memory for local variable

; set a as the min value
mov     eax, a_param
mov     min_loc, eax
; if b < min => min = b
mov     eax, b_param
cmp     eax, min_loc
jnl     next1
mov     min_loc, eax
next1:  mov     eax, c_param
        cmp     eax, min_loc
        jnl     next2
        mov     min_loc, eax
next2:  mov     eax, min_loc

; epilogue
mov     esp, ebp        ; destroy local variable
pop     ebp
ret
```

- To compile the above files using 32-bit GCC, type:

```
nasm -fwin32 minthree.asm
gcc -c main.c
gcc main.o minthree.obj -o main
```

- The first command, if success, generates `minthree.obj` object file
- The second command, if success, generates `main.o` object file
- The third command, if success, links two object files and generates an executable file named `main.exe`.

2.3.2 STDCALL Calling Convention

The `stdcall` is the standard calling convention for the Microsoft Win32 API.

- **Argument Order:** Same as `cdecl` (right to left)
- **Return Value:** Same as `cdecl` (EAX)
- **Stack Cleanup:** The callee cleans the stack before the function call returns. The callee does this by using a special form of the `ret` instruction. See the example below.

The only difference (between `cdecl` and `stdcall`) is who cleans the stack.

Example 2.4 (AddTwo Procedure).

AddTwo:

```
%define x_param DWORD [ebp + 8]
%define y_param DWORD [ebp + 12]

    push    ebp
    mov     ebp, esp
    mov     eax, y_param
    add     eax, x_param
    pop     ebp
    ret     8           ; clean up the stack
```

The caller's code would now be simpler, as it no longer needs to clean the stack:

```
push    6
push    5
call    AddTwo
; No "add esp, 8" needed!
```

2.4 LEA Instruction

Syntax

```
LEA    <destination>, <source>
```

- This instruction computes the effective address of the source operand and stores it in the destination operand.

Rules

- The source operand must be a memory address specified with one of the processors addressing modes
- The destination operand is general-purpose register.

- The valid forms are:

```
LEA    reg16, mem
LEA    reg32, mem
```

- The source operand can be in:
 - base addressing mode
 - base+displacement addressing mode
 - index addressing mode
 - base+index addressing mode

- base+index+displacement addressing mode
- This instruction is quite useful to get the address of a stack parameter.

Example 2.5 (Initialize an array). The following code segment, which is inside a function, declares a local array of characters named `myString` and set each element in the array to character `'*'`, as listed below:

```
void formatArray ()
{
    char myString[30];
    int i;

    for (i=0; i<30; i++)
        myString[i] = '*';
    .
    .
    .
}
```

The equivalent code in assembly language is listed below:

```
formatArray:
    push    ebp
    mov     ebp, esp
    sub     esp, 32      ; allocate 32 bytes

    lea     esi, [ebp - 32]
    mov     ecx, 30
top:    mov     BYTE [esi], '*'
    inc     esi
    loop    top

    .
    .
    .
```


Remarks

1. Although the array size is only 30 bytes, the ESP register is decremented by 32 to keep it **aligned** on a doubleword boundary.
2. Notice how LEA instruction is used to assign array's address to ESI, which is equivalent to assign ESI to the value of $EBP - 32$.
3. Without LEA, we can let ESI points to the array using the following code:

```
mov     esi, ebp
sub     esi, 32
```

4. LEA is more efficient than the above code, since these two instructions can be executed by LEA in one CPU cycle or less.

2.4.1 Flags Affected

- None.

2.5 ENTER and LEAVE Instructions

The CPU provides two instructions, ENTER and LEAVE, to automate the creation and destruction of stack frames. They are hardware-optimized shortcuts for the standard prologue and epilogue.

Syntax

```
ENTER    <numbytes>, <nestinglevel>
```

```
LEAVE
```

- The ENTER instruction automatically creates a stack frame for a called procedure.

- It performs three actions:
 1. Pushes EBP onto the stack (`push ebp`)
 2. Set EBP to the base of the stack frame (`mov ebp, esp`)
 3. Reserve space for local variables (`sub esp, numbytes`)
- Both operands must be immediate values. **Numbytes** specifies the number of bytes of stack space to reserve for local variables. **Nestinglevel** specifies the lexical nesting level of the procedure (read Intel document).
- **Numbytes** is always roundup to a multiple of 4 to keep ESP on a double-word boundary.
- The LEAVE instruction terminates the stack frame for a procedure.
 - It reverse the action of a previous ENTER instruction by:
 1. restoring ESP (`mov esp, ebp`), and
 2. restoring EBP to the value they were assigned when the procedure was called (`pop ebp`).

Example 2.6 (Revisit AddTwo Procedure).

AddTwo:

```
%define x_param DWORD [ebp + 8]
%define y_param DWORD [ebp + 12]

    enter    0, 0
    mov     eax, y_param
    add     eax, x_param
    leave
    ret
```

3 Recursion

3.1 Introduction

Recursion occurs when a procedure calls itself, either directly or indirectly. This powerful technique is essential for solving problems that can be broken down into smaller, similar subproblems.

3.2 Key Concepts

1. Base Case

Every recursive procedure must have a **base case** - a condition that stops the recursion. Without a base case, the procedure would call itself indefinitely, leading to **stack overflow**.

2. Recursive Case

The **recursive case** is where the procedure calls itself with modified parameters, moving closer to the base case with each call.

3.3 Recursive Factorial Example

Let's implement the factorial function in NASM. The factorial algorithm is defined as:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n - 1)! & \text{if } n > 0 \end{cases}$$

Here is the C implementation, which clearly shows the base case and the recursive step.

```

int factorial(int n) {
    if (n == 0) {          // Base case
        return 1;
    } else {               // Recursive step
        return n * factorial(n - 1);
    }
}

```

3.3.1 NASM Implementation

Now, let's implement this in assembly using the `cdecl` calling convention, where the caller is responsible for cleaning up the stack.

```

; Computes the factorial of a number n.
; Receives: [ebp + 8] = n
; Returns: EAX = n!
Factorial:
    push    ebp
    mov     ebp, esp

    mov     eax, [ebp + 8]      ; Get n
    cmp     eax, 0              ; Is n == 0? (Base case)
    jne     recursive_step     ; If not, jump to recursive step

    ; Base case: n is 0, so return 1
    mov     eax, 1
    jmp     epilogue

recursive_step:
    dec     eax                  ; eax = n - 1
    push    eax                  ; Push (n - 1) as argument for next
    call    Factorial            ; Call Factorial(n - 1)
    add     esp, 4               ; CALLER cleans stack (cdecl)

    ; This part runs AFTER the recursive call returns.

```

```

        ; EAX now holds the result of (n - 1)!
mov     ebx, [ebp + 8]      ; Get our original n back into EBX
mul     ebx                 ; EAX = EAX * EBX => (n-1)! * n

epilogue:
    pop     ebp
    ret                     ; Return to caller

```

3.4 Visualizing Recursion: The Stack for 3!

The best way to understand recursion is to see how the stack grows and shrinks. Let's trace the execution of `Factorial(3)`.

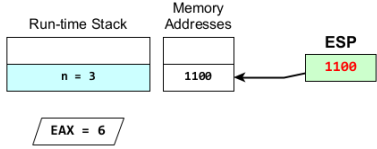
i Diagram Legend:

- Light Blue: Pushed by the **Caller** (arguments).
- Light Yellow: Pushed by the **CALL instruction** (return address).
- Light Green: Pushed by the Callee (procedure prologue).

Step	Description	Stack Frame
1	The main procedure begins by pushing its argument (<code>n=3</code>) and calling <code>Factorial</code> . The <code>CALL</code> instruction then pushes the return address. Finally, the <code>Factorial</code> prologue saves <code>main</code> 's <code>EBP</code> .	The diagram illustrates the stack state at Step 1. The 'Run-time Stack' is shown as a vertical container with three frames. The top frame is light blue and contains 'n = 3'. The middle frame is light yellow and contains 'Return to main()'. The bottom frame is light green and contains 'main()'s EBP'. To the right, a 'Memory Addresses' column lists the addresses 1100, 1096, and 1092. An 'ESP' register box shows the value 1092, with an arrow pointing to the bottom of the stack frame.

Step	Description	Stack Frame
2	Inside Factorial, n=3 is not 0. The procedure pushes its argument for the next call (n=2), calls itself, and a new stack frame is created.	Run-time Stack n = 3 Return to main() main()'s EBP n = 2 Return to Fact(3) Fact(3)'s EBP Memory Addresses 1100 1096 1092 1088 1084 1080 ESP 1080
3	The process repeats. n=2 is not 0, so Factorial pushes n=1 and calls itself again, creating a third stack frame.	Run-time Stack n = 3 Return to main() main()'s EBP n = 2 Return to Fact(3) Fact(3)'s EBP n = 1 Return to Fact(2) Fact(2)'s EBP Memory Addresses 1100 1096 1092 1088 1084 1080 1076 1072 1068 Stack Frame of Factorial 3 Stack Frame of Factorial 2 Stack Frame of Factorial 1 ESP 1068
4	Still not at the base case. Factorial pushes n=0 and calls itself one last time. The stack is now at its deepest point.	Run-time Stack n = 3 Return to main() main()'s EBP n = 2 Return to Fact(3) Fact(3)'s EBP n = 1 Return to Fact(2) Fact(2)'s EBP n = 0 Return to Fact(1) Fact(1)'s EBP Memory Addresses 1100 1096 1092 1088 1084 1080 1076 1072 1068 1064 1060 1056 Stack Frame of Factorial 3 Stack Frame of Factorial 2 Stack Frame of Factorial 1 Stack Frame of Factorial 0 ESP 1056

Step	Description	Stack Frame																																	
5	Inside Factorial(0), n is finally 0. The procedure hits its base case, sets EAX = 1, and returns. The stack now begins to unwind.	Run-time Stack <table border="1"> <thead> <tr> <th>Stack Frame</th> <th>Memory Address</th> <th>Content</th> </tr> </thead> <tbody> <tr> <td>Stack Frame of Factorial 3</td> <td>1100</td> <td>n = 3</td> </tr> <tr> <td></td> <td>1096</td> <td>Return to main()</td> </tr> <tr> <td></td> <td>1092</td> <td>main()'s EBP</td> </tr> <tr> <td>Stack Frame of Factorial 2</td> <td>1088</td> <td>n = 2</td> </tr> <tr> <td></td> <td>1084</td> <td>Return to Fact(3)</td> </tr> <tr> <td></td> <td>1080</td> <td>Fact(3)'s EBP</td> </tr> <tr> <td>Stack Frame of Factorial 1</td> <td>1076</td> <td>n = 1</td> </tr> <tr> <td></td> <td>1072</td> <td>Return to Fact(2)</td> </tr> <tr> <td></td> <td>1068</td> <td>Fact(2)'s EBP</td> </tr> <tr> <td></td> <td>1064</td> <td>n = 0</td> </tr> </tbody> </table> ESP: 1064 EAX = 1	Stack Frame	Memory Address	Content	Stack Frame of Factorial 3	1100	n = 3		1096	Return to main()		1092	main()'s EBP	Stack Frame of Factorial 2	1088	n = 2		1084	Return to Fact(3)		1080	Fact(3)'s EBP	Stack Frame of Factorial 1	1076	n = 1		1072	Return to Fact(2)		1068	Fact(2)'s EBP		1064	n = 0
Stack Frame	Memory Address	Content																																	
Stack Frame of Factorial 3	1100	n = 3																																	
	1096	Return to main()																																	
	1092	main()'s EBP																																	
Stack Frame of Factorial 2	1088	n = 2																																	
	1084	Return to Fact(3)																																	
	1080	Fact(3)'s EBP																																	
Stack Frame of Factorial 1	1076	n = 1																																	
	1072	Return to Fact(2)																																	
	1068	Fact(2)'s EBP																																	
	1064	n = 0																																	
6	Execution returns to Factorial(1). EAX holds 1 (the result of 0!). The procedure now multiplies this by its own n (which is 1). The result in EAX is $1 * 1 = 1$. Factorial(1) then returns.	Run-time Stack <table border="1"> <thead> <tr> <th>Stack Frame</th> <th>Memory Address</th> <th>Content</th> </tr> </thead> <tbody> <tr> <td>Stack Frame of Factorial 3</td> <td>1100</td> <td>n = 3</td> </tr> <tr> <td></td> <td>1096</td> <td>Return to main()</td> </tr> <tr> <td></td> <td>1092</td> <td>main()'s EBP</td> </tr> <tr> <td>Stack Frame of Factorial 2</td> <td>1088</td> <td>n = 2</td> </tr> <tr> <td></td> <td>1084</td> <td>Return to Fact(3)</td> </tr> <tr> <td></td> <td>1080</td> <td>Fact(3)'s EBP</td> </tr> <tr> <td>Stack Frame of Factorial 1</td> <td>1076</td> <td>n = 1</td> </tr> </tbody> </table> ESP: 1076 EAX = 1	Stack Frame	Memory Address	Content	Stack Frame of Factorial 3	1100	n = 3		1096	Return to main()		1092	main()'s EBP	Stack Frame of Factorial 2	1088	n = 2		1084	Return to Fact(3)		1080	Fact(3)'s EBP	Stack Frame of Factorial 1	1076	n = 1									
Stack Frame	Memory Address	Content																																	
Stack Frame of Factorial 3	1100	n = 3																																	
	1096	Return to main()																																	
	1092	main()'s EBP																																	
Stack Frame of Factorial 2	1088	n = 2																																	
	1084	Return to Fact(3)																																	
	1080	Fact(3)'s EBP																																	
Stack Frame of Factorial 1	1076	n = 1																																	
7	Execution returns to Factorial(2). EAX now holds 1 (the result of 1!). It multiplies this by its own n (which is 2). The result in EAX is $1 * 2 = 2$. Factorial(2) then returns.	Run-time Stack <table border="1"> <thead> <tr> <th>Stack Frame</th> <th>Memory Address</th> <th>Content</th> </tr> </thead> <tbody> <tr> <td>Stack Frame of Factorial 3</td> <td>1100</td> <td>n = 3</td> </tr> <tr> <td></td> <td>1096</td> <td>Return to main()</td> </tr> <tr> <td></td> <td>1092</td> <td>main()'s EBP</td> </tr> <tr> <td>Stack Frame of Factorial 2</td> <td>1088</td> <td>n = 2</td> </tr> </tbody> </table> ESP: 1088 EAX = 2	Stack Frame	Memory Address	Content	Stack Frame of Factorial 3	1100	n = 3		1096	Return to main()		1092	main()'s EBP	Stack Frame of Factorial 2	1088	n = 2																		
Stack Frame	Memory Address	Content																																	
Stack Frame of Factorial 3	1100	n = 3																																	
	1096	Return to main()																																	
	1092	main()'s EBP																																	
Stack Frame of Factorial 2	1088	n = 2																																	

Step	Description	Stack Frame
8	Execution returns to Factorial(3). EAX holds 2 (the result of 2!). It multiplies this by its own n (which is 3). The result in EAX is $2 * 3 = 6$. Factorial(3) returns to main, which cleans up the stack. The final result is in EAX.	 The diagram illustrates the state of the stack and registers. On the left, the 'Run-time Stack' shows a frame for 'Factorial(3)' with 'n = 3'. To the right, 'Memory Addresses' shows a box at '1100'. An arrow points from the 'ESP' register, which contains '1100', to this box. Below the stack, a box indicates 'EAX = 6'.

3.5 Recursive Sum Example

This example sums integers from 1 to n. It is structurally identical to Factorial, but uses ADD and follows the stdcall convention, where the **callee** is responsible for cleaning up the stack.

```

; Calculates the sum of integers from 1 to n. (stdcall convention)
; Receives: [ebp + 8] = n
; Returns: EAX = sum
CalcSum:
    push    ebp
    mov     ebp, esp

    mov     eax, [ebp + 8]      ; Get n
    cmp     eax, 0             ; Base case: is n == 0?
    je      base_case

    ; Recursive step: return n + CalcSum(n-1)
    dec     eax                ; eax = n - 1
    push    eax                ; Push argument for CalcSum(n-1)
    call    CalcSum            ; EAX now holds sum for (n-1)

```



```

        ; After call returns, add our n to the result
mov     ebx, [ebp + 8]      ; Get our n back
add     eax, ebx           ; EAX = (sum of n-1) + n
jmp     epilogue

base_case:
        mov     eax, 0      ; Sum of 0 is 0

epilogue:
        pop     ebp
        ret     4           ; CALLEE cleans stack (stdcall)

```

3.6 Integer to Binary String Example

A common task in programming is to display the binary representation of a number. For example, how can a program convert the integer 26 (stored in memory as 00011010b) into the printable character string "11010"? The standard algorithm uses repeated division and remainders:

- $26 / 2 = 13$, remainder **0**
- $13 / 2 = 6$, remainder **1**
- $6 / 2 = 3$, remainder **0**
- $3 / 2 = 1$, remainder **1**
- $1 / 2 = 0$, remainder **1**

Reading the remainders from bottom-to-top gives us the correct binary string: 11010. The challenge for a program is this reversal. The first digit we calculate (0) is the *last* digit we need to print.

This is a perfect scenario for recursion! The stack's LIFO (Last-In, First-Out) nature provides a natural way to reverse the order of the digits.

3.6.1 The Recursive Strategy

Think of it like leaving a trail of breadcrumbs as you venture into a cave:

1. **The Recursive Step (Going Deeper):** For any number n greater than 0, we first figure out the *next* part of the number ($n / 2$) and leave a breadcrumb behind for the *current* digit ($n \% 2$). We then recursively call the function to venture deeper with $n / 2$. We don't print anything yet.
2. **The Base Case (The End of the Cave):** When n is 0, we've gone as deep as we can. We simply turn around and go back.
3. **The Unwinding (Following the Trail Back):** As each recursive call returns, we "pick up" the breadcrumb we left behind (the remainder) and print it. Since we are returning from the deepest call first, we naturally print the digits in the correct, reversed order.

```
#include <stdio.h>

void print_binary(unsigned int n) {
    // Base Case: If n is 0, we've gone as deep as we can. Stop.
    if (n == 0) {
        return;
    }

    // Recursive Step: Call the function for the rest of the number first
    print_binary(n / 2);

    // Unwinding Step: This code runs AFTER the recursive call returns.
    // It prints the breadcrumb left behind by the current level.
    int remainder = n % 2;
    putchar(remainder + '0'); // Convert 0 or 1 to '0' or '1'
}
```

3.6.2 Tracing print_binary(26)

Call Stack (Depth)	Calculation (n / 2)	Breadcrumbs	Action
print_binary(26)	26 / 2 = 13	rem = 0	Save 0, call print_binary(13)
print_binary(13)	13 / 2 = 6	rem = 1	Save 1, call print_binary(6)
print_binary(6)	6 / 2 = 3	rem = 0	Save 0, call print_binary(3)
print_binary(3)	3 / 2 = 1	rem = 1	Save 1, call print_binary(1)
print_binary(1)	1 / 2 = 0	rem = 1	Save 1, call print_binary(0)
print_binary(0)		-	Base case reached. Return.

Now, the stack unwinds and the putchar calls execute:

Unwinding Step	Pick up Breadcrumbs	Action	Output
print_binary(1)	rem = 1	Putchar (1 + '0')	1
print_binary(3)	rem = 1	Putchar (1 + '0')	11
print_binary(6)	rem = 0	Putchar (0 + '0')	110
print_binary(13)	rem = 1	Putchar (1 + '0')	1101
print_binary(26)	rem = 0	Putchar (0 + '0')	11010

3.6.3 The Assembly Implementation

Here is a complete, runnable program demonstrating the print_binary procedure.

```

        global _main
        extern _putchar

        section .text

;-----
; print_binary(n)
; Prints the binary representation of an unsigned integer.
; Receives: [ebp + 8] = n (the number to print)
; Clobbers: EAX, EDX
;-----
print_binary:
        push    ebp
        mov     ebp, esp

        mov     eax, [ebp + 8]    ; Get our number, n
        cmp     eax, 0            ; Base case: if n is 0, we're done
        je      end_recursion

        ; --- Recursive Step ---
        mov     edx, 0            ; Clear EDX for division
        mov     ebx, 2            ; Divisor is 2
        div     ebx               ; EAX = n / 2, EDX = n % 2
        push    edx               ; store remainder

        push    eax               ; Push the result of n / 2
        call    print_binary      ; Recurse with PrintBinary(n / 2)
        add     esp, 4            ; Clean up stack (cdecl)

        ; --- Unwinding Step ---
        ; This code executes AFTER the recursive call returns.
        ; The remainder (0 or 1) is still in EDX from our division.
        pop     edx               ; restore remainder
        add     edx, '0'          ; Convert 0 -> '0', 1 -> '1'

        push    edx               ; Push the character to print
        call    _putchar
        add     esp, 4

```

```

end_recursion:
    pop     ebp
    ret

;-----
; Main program entry point
;-----
_main:
    push    26                ; Test with the number 26
    call    print_binary
    add     esp, 4

    ; print a newline character for clean output
    push    10                ; ASCII code for newline
    call    _putchar
    add     esp, 4

    xor     eax, eax          ; Return 0
    ret

```